实时操作系统RT-Thread启动流程剖析

原创 赵剑川 等 嵌入式系统专家之声 2026年8月19日 08:00 北京

点击上方"蓝字" 关注我们吧！

嵌入式系统专家之声
嵌入式系统专家之声依托嵌入式系统联谊会专家团队，汇集嵌入式系统产、学、研和媒体...
308篇原创内容

公众号

2026.08.19

本文字数：5871

预计阅读时长：15分钟

摘要

RT-Thread是一个物联网实时操作系统（RTOS），它提供了线程调度、时钟管理、内存管理、线程同步与通信、中断管理等基本功能要素，具有组件完整丰富、高度可伸缩、简易开发、超低功耗、高安全性等特点。在深入分析RT-Thread的功能要素和函数调用关系基础上，简要给出了其在STM32单片机芯片上的启动流程，重点剖析了RT-Thread的芯片上电、板级初始化、定时器初始化、调度器初始化、线程创建、线程切换等流程，可作为RT-Thread在不同微控制器上移植的技术基础，也可作为分析其他RTOS启动过程的技术参考。

关键词

RT-Thread；STM32；启动流程；线程调度

作者：赵剑川，赵静，欧启标，张宇，张检保

作者单位：广东机电职业技术学院，广州 510000

中图分类号：TP316.2

文献标识码：A

0 引言

RT-Thread是一款主要由中国开源社区主导开发的应用于物联网的开源嵌入式实时操作系统，它支持多线程管理及其调度、线程同步与通信、时钟管理、内存管理、中断管理等基本要素。同时包含多种操作系统常用组件和服务，如虚拟文件系统、FinSH命令行界面、网络框架、设备框架等。经过11年的累积发展，RT-Thread被广泛应用于能源、车载、医疗、消费电子等多个行业。基于

RTOS的嵌入式开发对应用系统的稳定性、实时性和启动时间等都有严格的要求，在启动过程中要完成栈空间、堆空间、线程栈大小、时间嘀嗒、线程调度机制等方面的设置[1]，具有启动时间短且复杂性高的特点。因此，理解和掌握RTOS的启动过程，对开发人员设计出响应速度快、稳定性强的嵌入式系统很有意义。目前，有关操作系统启动的研究集中在Android操作系统[2]、MQX操作系统[3]、μC/OS-III操作系统[4]、Linux操作系统[5]、MTX操作系统[6]以及ARM嵌入式系统的启动过程[7]等方面，但对RT-Thread启动流程方面的研究缺乏相关资料。为此本文采用RT-Thread Studio开发环境，基于STM32控制器对RT-Thread实时操作系统的启动流程进行分析，剖析其从芯片上电启动到rtthread_srartup函数，最终进入RT-Thread的启动全过程，结合关键代码、宏定义函数、程序流程图、PendSV中断等分析其实现的原理，可为RT-Thread的应用研究以及在其他微控制器上的移植提供依据，也可为分析其他RTOS的启动过程提供技术参考。

1 RT-Thread启动过程

RT-Thread启动过程可以分为两个阶段，分别是芯片上电启动阶段和RT-Thread实时内核启动阶段。如图1所示，芯片上电启动阶段包括栈指针和PC指针初始化、进入复位中断向量、从Flash中复制初始化数据到RAM、清空bss数据段、初始化时钟、跳转至RT-Thread实时内核启动。当跳转到entry()后，系统进入RT-Thread实时内核启动阶段，此阶段会初始化必要的外设、创建必要线程并启动线程调度器，完成操作系统的主要工作。

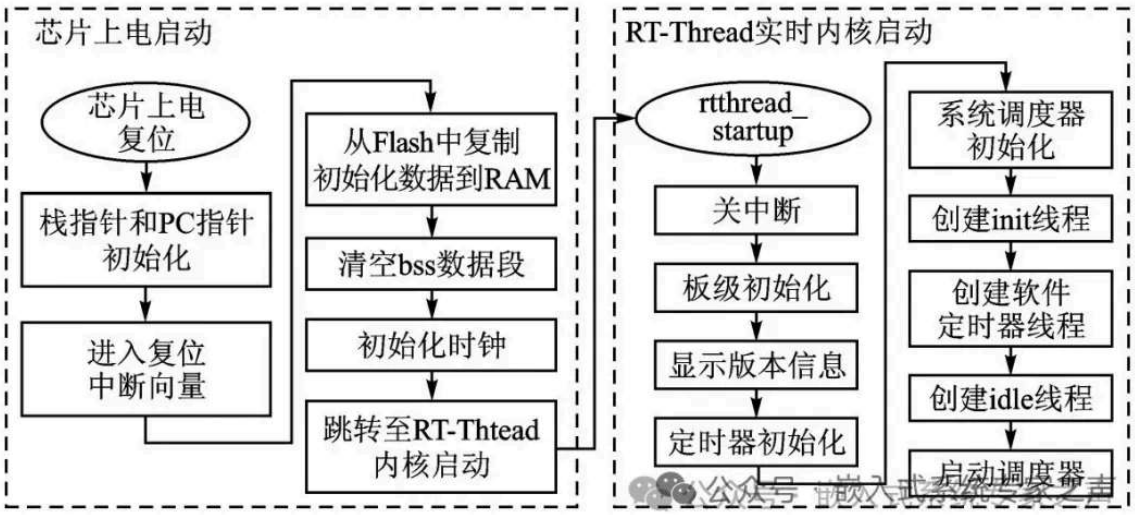


图1 RT-Thread启动过程

芯片上电启动阶段的代码在startup_stm32f103xb.s文件中实现，文件中定义了中断服务向量表的代码段为.isr_vector，在链接文件link.lds中又把代码段．isr_vector链接到了ROM的起始位置。而STM32F103RB的内部ROM起始地址为0x0800 0000，大小为128KB。在Flash启动模式下，STM32F103RB芯片复位后，芯片内部机制会自动从0x0800 0000地址读取栈指针到MSP，从0x0800 0004读取中断地址到PC，由于0x0800 0004存储了函数Reset_Handler的首地址，所以STM32芯片复位会从Reset_Handler开始执行。Reset_Handler执行从Flash中复制初始化数据到RAM、清空bss数据段、初始化系统时钟等一系统操作，最后跳转至rt-thread/src/components.c文件中定义的entry()函数，entry()函数通过调用rtthread_startup()函数完成RT-Thread实时内核的启动。

2 RT-Thread实时内核启动流程分析

RT-Thread实时内核启动的第二阶段由rt-thread/src/components.c文件中的rtthread_startup()函数完成，此阶段的任务主要包括板级初始化、定时器初始化、系统线程调度器初始化、创建init线程线程、创建软件定时器线程、创建空闲线程，最后启动线程调度器进行实时调度。

2.1 板极初始化

板级初始化的任务是驱动硬件板的主要外设，属于外设驱动。RT-Thread操作系统对于STM32芯片的板级初始化主要包括时钟树配置、滴嗒时钟初始化和串口初始化。其中时钟树配置主要配置系统总线的时钟源及各外设的时钟频率，滴嗒时钟是操作系统的时间基准，而串口是系统的人机交互接口。

2.1.1 时钟树配置

时钟树配置在drivers/drv_clk.c文件中实现，其配置代码由system_clock_config()函数实现，可以通过修改该函数来完成配置。但在RT-Thread Studio开发环境中，也可以很方便地通过双击项目树的CubeMX Settings选项用CubeMX软件的图形界面进行配置，通过CubeMX图形配置后，时钟树配置函数会自动更新为SystemClock_Config()。

2.1.2 滴嗒时钟

滴嗒时钟初始化在drivers/drv_common.c文件的rt_hw_systick_init()函数中实现。该函数调用ST公司开发的HAL驱动库接口函数HAL_SYSTICK_Config()，默认把滴嗒时钟初始化为每1ms产生一次中断，也就是说RT-Thread系统的时钟滴嗒为1ms，此值是可以通过rtconfig.h的RT_TICK_PER_SECOND宏进行配置的，滴嗒时钟的中断优先级配置为最高优先级0。

2.1.3 串口

串口的初始化在drivers/drv_uart.c文件实现。首先，该文件定义了两个关键数组：uart_config和uart_obj，其中uart_config用于描述每个uart设备的硬件资源配置，如相关操作寄存器和接收发送引脚信息等；uart_obj描述了每个uart设备的驱动接口，如接收和发送函数等。其次，通过函数rt_hw_usart_init()实现对串口的初始化，该函数最终调用函数rt_hw_serial_register()向RT-Thread的设备驱动框架层注册串口设备。

2.2 硬件中断模式定时器初始化

RT-Thread的定时器是通过队列来管理的，系统中有两种定时器模式，分别是硬件中断模式和软件线程模式，相应有两个定时器队列。硬件中断模式定时器在滴嗒时钟中断上下文中轮询定时器队列并执行到期定时器所对应的回调函数；而软件线程模式定时器是在线程上下文中轮询定时器队列并执行到期定时器所对应的回调函数。此阶段的初始化只是对硬件中断模式定时器队列进行初始化。RT-Thread为了提高定时器队列轮询速度，通过跳表数据结构变量rt_timer_list来管理定时器。定时器跳表结构如图2所示。

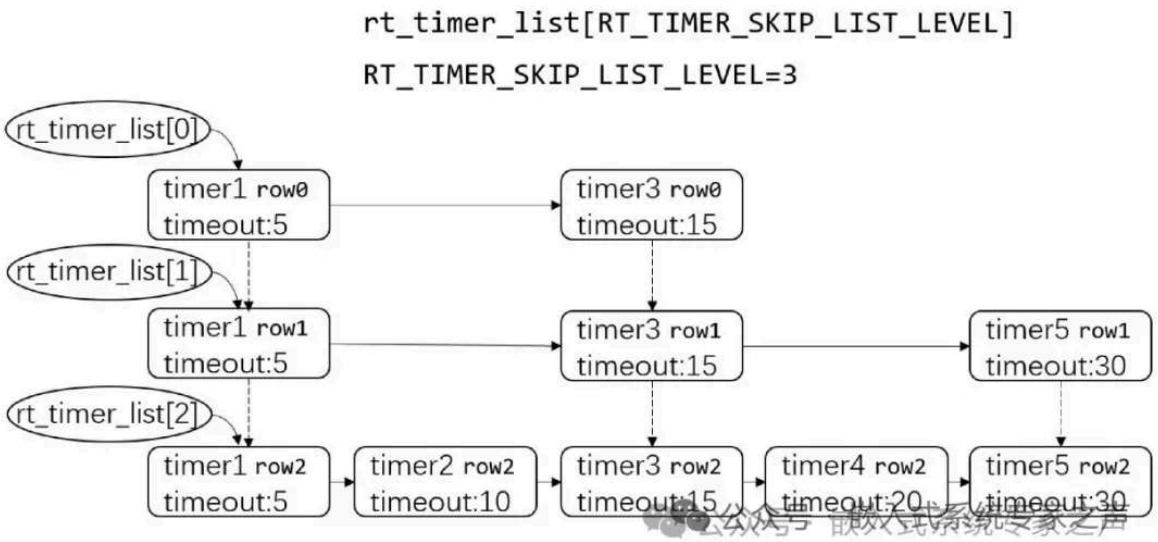


图2 定时器跳表结构

2.3 线程调度器初始化

调度器完成操作系统的线程切换功能，RT-Thread线程调度器的初始化代码在scheduler.c中的rt_system_scheduler_init()函数中实现，主要初始化了两个线程队列，分别是就绪线程队列和僵尸线程队列。RT-Thread为了提高线程切换速度，把就绪队列按优先级分为多个子优先级队列，默认情况下系统支持32个线程优先级，用数组变量rt_thread_priority_table存放32个优先级队列的队列头。另外，用变量rt_thread_ready_priority_group记录当前系统中所有就绪线程中具有最高优先级的线程编号。这样，调度器在切换线程时，可以根据此编号在rt_thread_priority_table中快速定位到最高优先级线程，进而从队列取出并运行线程。此阶段，调度器只是做了初始化，还未真正开始工作。

2.4 创建init线程

RT-Thread的init线程就是用户写的main函数对应用的线程，它的初始化在components.c的rt_application_init()函数实现。init线程的入口函数是main_thread_entry()，从以下代码可以看出，此入口函数其实是对main函数的封装。init线程的优先级别在初始化时设置为10，可以通过修改宏RT_MAIN_THREAD_PRIORITY来实现。

```
1  void rt_application_init(void){
2    rt_thread_t tid;
3    /*创建线程*/
4    tid=rt_thread_create (" main ", main_thread_ entry, RT_NULL,RT_MAIN_THREAD_S
5    rt_thread_startup(tid);//唤醒线程，即把线程插入到就绪队列
6  }
```

2.5 创建软件定时器线程

软件线程模式定时器需要在线程上下文中轮询定时器队列并执行到期定时器所对应的回调函数，所以系统必须创建一个专用线程来完成此功能。线程的初始化在timer.c的rt_system_timer_thread_init()函数实现，该函数先初始化软件线程模式定时器跳表rt_soft_timer_list,再创建定时器线程，入口函数为rt_thread_timer_entry(),其程序流程图如图3所示。

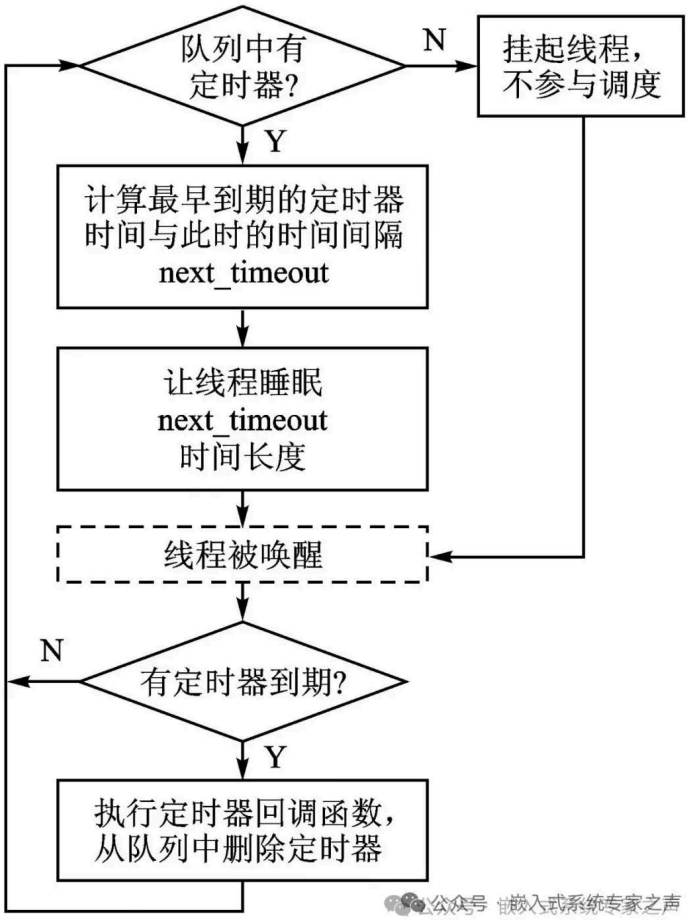


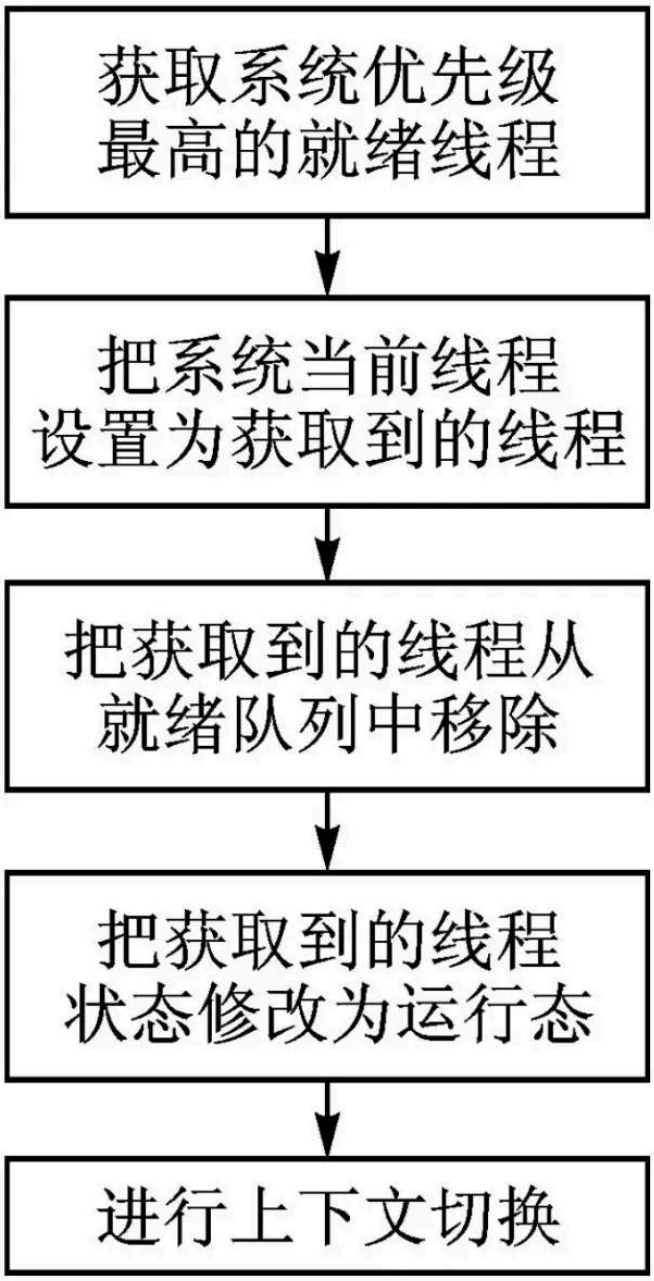
图3 软件线程模式定时器执行流程

2.6 创建空闲线程

空闲线程对操作系统来说也是必不可少的线程，它通常用于系统空闲时回收一些系统资源。空闲线程的创建代码在idle.c文件的rt_thread_idle_init()函数中实现。空间线程的入口函数为rt_thread_idle_entry()，它的主要作用是回收僵尸线程。

2.7 启动线程调度器

前面创建的3个线程在没有启动调度器前是不会被运行的，在操作系统启动的最后阶段，需要启动调度器，由调度器完成线程的调度工作。线程调度器的实现代码在scheduler.c中实现，其中rt_system_scheduler_start()函数用于开启线程调度器，其执行流程如图4所示。



公众号 嵌入式系统专家之声

图4 调度器启动流程

由图4可知，系统从就绪队列中选出优先级最高的线程，把它设置为当前线程，同时把线程状态修改为正在运行中，最后调用rt_hw_context_switch_to()函数进行上下文切换，切换完成后，CPU就进入到该线程执行。

获取系统优先级最高的就绪线程的方法如下：用32位变量rt_thread_ready_priority_group记录当前系统中就绪线程的优先级分布，32位对应32个优先级别。这时通过函数_ _rt_ffs()得到当前系统中最高优先级线程的优先级编号。该函数可以快速返回参数中第1个值为1的位的编号，比如参数如果是26(二进制表示为10110)，则从低位起，第1个为1的bit位排第2，则函数返回2，由此可知，系统中当前最高优先级线程的优先级编号为2-1=1。

上下文切换函数rt_hw_context_switch_to()的主要功能是保存当前线程的栈地址，然后触发PendSV中断进入PendSV_Handler函数，在PendSV_Handler函数中完成线程切换，整个切换代码流程图如图5所示。

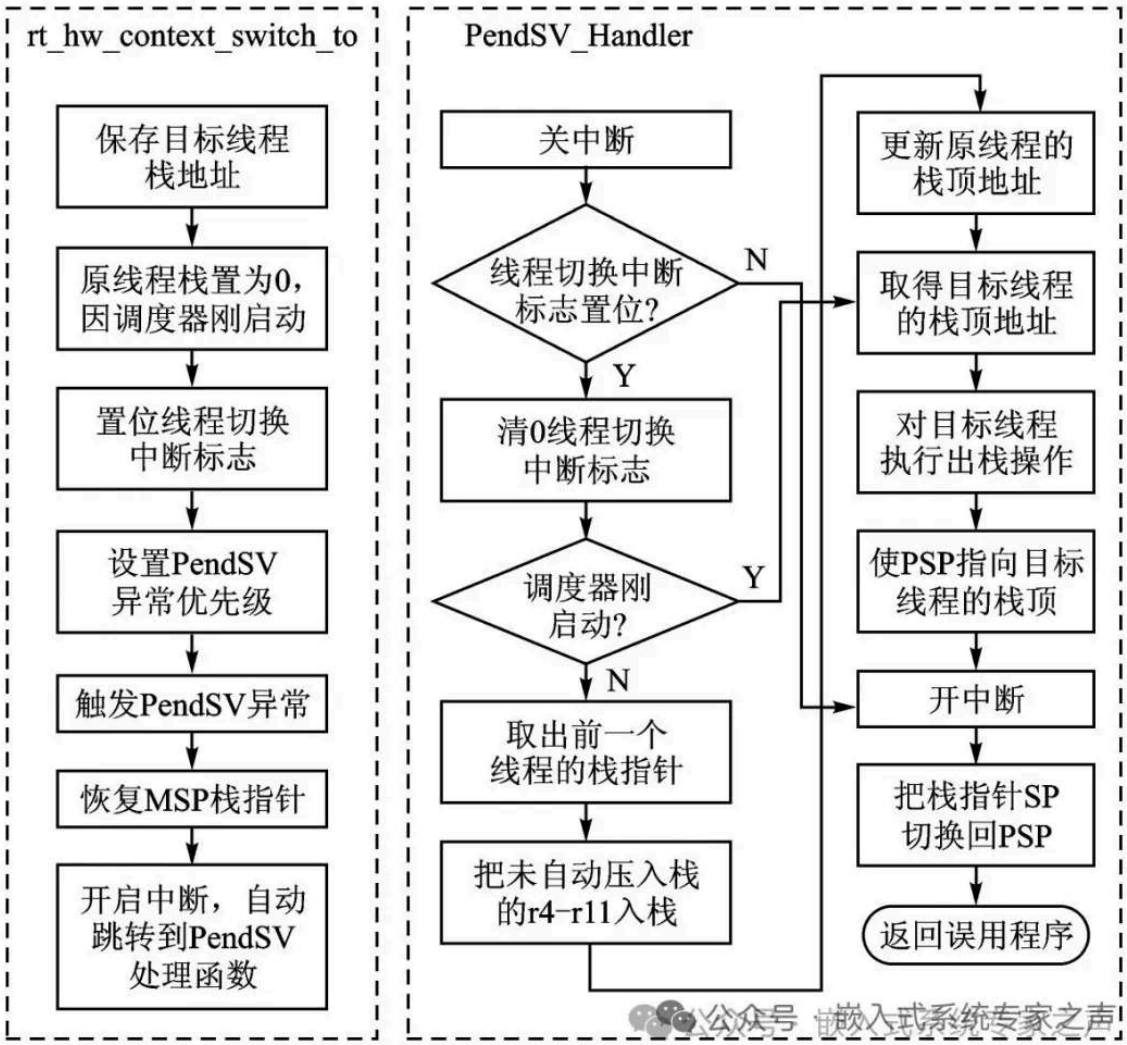


图5 线程切换流程图

线程切换的实质就是线程栈空间的切换，即从原来运行线程的栈空间切换到目的线程的栈空间，代码在PendSV中断服务程序中实现线程切换，刚好可以在进入中断服务和退出中断服务时由硬件自动完成现场保护和恢复的必要工作。

3 结语

RT-Thread操作系统的启动流程是一个极其复杂的过程，涉及到系统时钟的设置、硬件平台必要的外设驱动、系统定时器的管理、系统线程调度器管理、必要线程的创建和执行。涉及代码包括汇编和C语言，其函数调用关系也非常复杂。本文通过RT-Thread Studio开发工具对RT-Thread操作系统启动流程进行分析，简要给出了芯片上电启动过程，通过源码、宏定义函数、PendSV中断、流程图等方式来着重剖析RT-Thread的启动过程。通过剖析，有助于读者快速理解RT-Thread的启动流程、调度机制和整体架构，为简化启动流程、优化执行过程、提升启动速度等进一步研究工作提供研究基础，也为RT-Thread在不同微控制器上的移植提供了技术基础。

参考文献

[1]王宜怀，朱仁浪，姚望舒，嵌入式实时操作系统MQX应用开发技术——ARM Cortex-M微处理器[M]．北京：电子工业出版社，2014:39-52.
[2]刘博文，顾乃杰，谷德贺，等，移动平台Android操作系统虚拟化技术的实现[J]．计算机工程与应用，2017,53(14):37-43.
[3]蒋建武，王宜怀．MQX操作系统启动流程剖析[J]．福州大学学报（自然科学版），2016,44(2):64-68.
[4]乔焱，陈媛，贾家宁，等．C/OS-III操作系统的优化[J]．兵工自动化，2019,38(3):31-33.
[5]冯瑞青，张激，赵俊才．异构处理器多操作系统协同技术研究[J]．计算机系统应用，2018,27(12):92-97.

（本文由《单片机与嵌入式系统应用》杂志社授权发表，原文刊发在2022年第6期）

相/关/推/荐

实时操作系统RT-Thread Smart 在STM32MP1上的移植

RusT-Thread：基于Rust面向资源受限嵌入式设备的操作系统的实践

专家观点：嵌入式实时操作系统的真正创新

嵌入式系统专家之声推荐搜索

RTOS | 物联网 | RT-Thread

【点击上方👆搜索词条可查看号内更多其他内容】



关注我们，了解更多精彩内容

转发，点赞，在看，安排一下?

上一篇 · 基于FPGA的类脑计算平台通用系统架构设计